Static Members in Class Context

The special variable `$this` is made available in an object's methods, and it's how we access the object's other members, using the object operator `->` or scope resolution operator `::` depending on the context of the target members. The problem with static methods is that there is nothing for `$this` to reference: no object has been instantiated, so there cannot be a `$this` when executing in class context. So, instead of `$this`, we use the `self` keyword. `self` provides access to the class context members:

```
    public static function getQuote(): string
    {
        return self::$quoteStash[rand(0,1)];
    }
```

Here we have a static method `getQuote()`, it has to use `self::$quoteStash` to access the static `$quoteStash` class property.

Static methods are restricted to other static members: it isn't possible to call a non-static method or reference a non-static property, and doing so will cause a fatal error:

```
class QuoteBot
{
    private function getInsight(): string
    {
        return "I've always found that writing comes from a great inner
        pain." . PHP_EOL;
    }

    public static function insight(): void
    {
        print self::getInsight();
    }
}

QuoteBot::insight();
```

```
Output:
Fatal error: Uncaught Error: Non-static method QuoteBot::getInsight()
cannot be called statically
```